Backend AI Engineering Portfolio — Content Plan

Name: Sibgha Mursaleen
Date: 16 July 2026

1. Positioning Statement

"I design and build backend systems—from database to deployment—using modern engineering practices."

Why this claim: Your strongest asset is range Python/FastAPI and C#/ASP.NET Core in the same portfolio signals adaptability, not just "I learned one framework." "From database to deployment" points directly at evidence a reviewer can check: Docker Compose, schema design, query optimization, API design. It doesn't claim anything that can't be verified by opening a repo.

2. Site Structure & Content Map

Home

- Headline: positioning statement
- Sub-line: what you do "APIs, databases, and backend systems — Python and C#"
- Featured project snapshot: **Task API (Postgres, Redis, Docker)** link to full case study
- Stack strip: FastAPI, PostgreSQL, Redis, Docker, ASP.NET Core, Entity Framework
- **CTA:** "See my work" → Projects

Projects (ranked order — see Section 3 for evidence)

1. Task API: Dockerized with Postgres, Redis & Query Optimization
2. SIBA Complaint Management System
3. E-commerce Full Stack
4. VNIAS-IJILS Backend API

Each project shown as a card: name, one-line problem statement, stack tags, link to case study. **CTA per card:** "Read the case study"

Project Case Studies (one page per project, same structure each time)

- The problem: what needed to be built and why
- Your role and the stack
- Architecture (diagram if available)

- One real challenge and how you solved it
- Outcome: what it does now, and its actual state (live / local-only / in progress)
- Links: GitHub repo, live demo if it exists
- **CTA:** "View the code" (GitHub link) + "Next project"

About

- Anchored to the claim, not generic what got you into backend engineering, what you focus on now, what you're building toward
- One line on stack range: Python and C#, not just one ecosystem
- **CTA:** "Get in touch" or "View my resume"

Contact

- Email, GitHub, LinkedIn, resume download
- Kept short this page's only job is removing friction, not persuading

Structural principle: every page either reinforces the claim or removes friction toward one action a reviewer reaching out or opening your GitHub. Nothing exists just to look complete.

3. Project Ranking — Scored on Real Repo Evidence

Each project scored 1–5 on: Technical Complexity, Backend Relevance, Recruiter Impact, Demonstrated Engineering Skill. All four repos were read directly (READMEs, structure, and implementation details) nothing here is guessed.

Rank	Project	Complexity	Backend Relevance	Recruiter Impact	Demonstrated Skill	Total
1	Task API — Postgres + Redis + Docker	4	5	4	5	**18**
2	SIBA Complaint Management System	4	4	4	4	**16**
3	E-commerce Full Stack	4	3	4	3	**14**
4	VNIAS-IJILS Backend API	3	5	3	3	**14**

1. Task API (Postgres + Redis + Docker): Three-container Docker Compose orchestration (app, Postgres, Redis) with health checks so the app doesn't start before its dependencies are ready. Persistence is proven, not claimed tasks survive a full container teardown and restart. The repository pattern was swapped from in-memory to Postgres without touching the API layer, validating the abstraction actually works. Includes a genuine query optimization exercise: EXPLAIN ANALYZE before and after adding a B-tree index, with real query plan output. This last piece is rare at student level and is your strongest single proof point.

2. SIBA Complaint Management System: Most complete of the four. Role hierarchy (Senior Admin / Staff / Student), domain-restricted signup, EF Core with real migrations and seeded demo credentials, a resolution workflow, and an "immutable resolution proof" snapshot feature — a real accountability design decision, not boilerplate. Tied to a real institution, which reads as built-for-a-need rather than an exercise.

3. E-commerce Full Stack: JWT auth with bcrypt, cart/wishlist state, admin dashboard, MongoDB. Solid, recognizable domain. Backend relevance is lower only because roughly half the repo is frontend work, which splits the story.

4. VNIAS-IJILS Backend API: Backend-relevance is a perfect 5 pure API, no frontend, and the README describes real patterns (repository pattern, dependency injection, async DB pooling, global exception handling, streaming file uploads). Held back on the other three criteria because models/ and some repositories/ are placeholder (.gitkeep), auth is explicitly stubbed, and the caching layer is a simulated (mocked) Redis rather than a live one. The architecture thinking is real; the implementation isn't finished yet.

4. Proof / Asset Checklist

Every project needs:

- Public repo confirmed (all four are public — verified)
- Clean README (all four already have strong READMEs — verified)
- At least one screenshot or terminal output showing it running

Task API (Postgres + Redis + Docker):

- Screenshot of Swagger docs (/docs)
- Screenshot or copy of the EXPLAIN ANALYZE before/after output — this is your best differentiator, make it visible, not buried in a README
- Short screen recording of the persistence test (create task → docker compose down → up → task still there) — this is a strong, easy proof to capture

SIBA Complaint Management System:

- Screenshots: student dashboard, admin resolution view, one resolved ticket with its snapshot
- Confirm whether this can be deployed for a live demo, or stays local-only with screenshots/recording

E-commerce Full Stack:

- Fix the README clone command — it still says your-username instead of your actual GitHub handle
- Live demo link if deployed, or a screen recording if not
- Confirm solo vs. team — if team, specify which backend parts are yours

VNIAS-IJILS Backend:

- **Decision needed (see Section 5):** finish stubbed models/auth before featuring it, or present it honestly as in-progress
- If presented as in-progress: architecture diagram showing what's built vs. planned
- Clarify real-world scope in one line for the case study (who is this platform for, is it in use)

Portfolio-wide:

- Resume as downloadable PDF
- No testimonials section — skip it rather than leave it empty at student stage